```
"""Model validation, overfitting control and hyperparameter
tuning.

Run:  python validation_project.py
Needs: numpy, scipy, matplotlib, scikit-learn
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.stats import randint
from sklearn.datasets import load_breast_cancer
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import GradientBoostingClassifier,
RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report,
roc_auc_score
from sklearn.model_selection import (GridSearchCV,
RandomizedSearchCV, StratifiedKFold,
                                     cross_val_score,
cross_validate, learning_curve,
                                     train_test_split,
validation_curve)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier

SEED = 42

# ----------------------------------------------------
------- 1. Data + split
```

```
# Lock the test set away FIRST. It is used exactly once, at
the very end.
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=SEED)
cv = StratifiedKFold(n_splits=5, shuffle=True,
random_state=SEED)
print(f"Train: {X_train.shape}  Test: {X_test.shape}
Positive rate: {y.mean():.2f}\n")


# ----------------------------------------------------------
------- 2. Validation + diagnosis
def diagnose(name, model):
    """Cross-validate and report the train/validation gap
(the overfitting signal)."""
    r = cross_validate(model, X_train, y_train, cv=cv,
scoring="roc_auc",
                       return_train_score=True)
    tr, va = r["train_score"].mean(),
r["test_score"].mean()
    print(f"{name:<30} train {tr:.3f} | CV {va:.3f} +/-
{r['test_score'].std():.3f} | gap {tr - va:+.3f}")


print("== Step 1: baseline and overfitting diagnosis (ROC
AUC, 5-fold CV) ==")
diagnose("Dummy (majority class)", DummyClassifier())
diagnose("Tree, unrestricted",
DecisionTreeClassifier(random_state=SEED))
diagnose("Tree, max_depth=3",
DecisionTreeClassifier(max_depth=3, random_state=SEED))
diagnose("Logistic, scaled", Pipeline([("sc",
StandardScaler()),
```

```
                                        ("clf",
LogisticRegression(max_iter=2000))]))
print("A big train-vs-CV gap means overfitting. A low score
on both means underfitting.\n")


# ----------------------------------------------------------
------- 3. Overfitting control
# 3a. Validation curve: one hyperparameter vs.
train/validation score.
depths = np.arange(1, 13)
tr_s, va_s =
validation_curve(DecisionTreeClassifier(random_state=SEED),
X_train, y_train,
                                param_name="max_depth",
param_range=depths, cv=cv,
                                scoring="roc_auc")
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(depths, tr_s.mean(1), "o-", label="train")
ax.plot(depths, va_s.mean(1), "o-", label="validation")
ax.fill_between(depths, va_s.mean(1) - va_s.std(1),
va_s.mean(1) + va_s.std(1), alpha=0.2)
ax.set(xlabel="max_depth", ylabel="ROC AUC",
title="Validation curve: decision tree")
ax.legend(); fig.tight_layout();
fig.savefig("validation_curve.png", dpi=150);
plt.close(fig)
print(f"== Step 2a: best max_depth by validation score =
{depths[va_s.mean(1).argmax()]} "
      f"(saved validation_curve.png)")

# 3b. Learning curve: does more data help, or is the model
the problem?
sizes, tr_l, va_l = learning_curve(
```

```
    Pipeline([("sc", StandardScaler()), ("clf",
LogisticRegression(max_iter=2000))]),
    X_train, y_train, cv=cv, scoring="roc_auc",
    train_sizes=np.linspace(0.1, 1.0, 8),
random_state=SEED)
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(sizes, tr_l.mean(1), "o-", label="train")
ax.plot(sizes, va_l.mean(1), "o-", label="validation")
ax.set(xlabel="training examples", ylabel="ROC AUC",
title="Learning curve: logistic regression")
ax.legend(); fig.tight_layout();
fig.savefig("learning_curve.png", dpi=150); plt.close(fig)
print("== Step 2b: learning curve saved
(learning_curve.png)")

# 3c. Regularization: smaller C = stronger penalty.
print("\n== Step 2c: regularization strength (logistic C)
==")
for C in [0.001, 0.01, 0.1, 1, 10, 100]:
    pipe = Pipeline([("sc", StandardScaler()), ("clf",
LogisticRegression(C=C, max_iter=2000))])
    s = cross_val_score(pipe, X_train, y_train, cv=cv,
scoring="roc_auc")
    print(f"  C={C:<7} CV AUC {s.mean():.3f} +/-
{s.std():.3f}")

# 3d. Early stopping: stop adding trees when the internal
validation score stalls.
gb = GradientBoostingClassifier(n_estimators=1000,
learning_rate=0.05, max_depth=3,
                                validation_fraction=0.15,
n_iter_no_change=10,

random_state=SEED).fit(X_train, y_train)
```

```
print(f"\n== Step 2d: early stopping used
{gb.n_estimators_} of 1000 trees ==\n")


# ------------------------------------------------------
------- 4. Hyperparameter tuning
# The scaler lives INSIDE the pipeline so it is re-fit on
each training fold (no leakage).
print("== Step 3: hyperparameter tuning ==")
log_search = GridSearchCV(
    Pipeline([("sc", StandardScaler()), ("clf",
LogisticRegression(max_iter=2000))]),
    {"clf__C": np.logspace(-3, 2, 11)},
    cv=cv, scoring="roc_auc", n_jobs=-1).fit(X_train,
y_train)
print(f"Grid search (logistic): best
{log_search.best_params_}, CV AUC
{log_search.best_score_:.3f}")

rf_space = {"n_estimators": randint(100, 300), "max_depth":
randint(2, 12),
            "min_samples_leaf": randint(1, 10),
"max_features": ["sqrt", "log2", 0.5]}
rf_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=SEED), rf_space,
n_iter=15, cv=cv,
    scoring="roc_auc", n_jobs=-1,
random_state=SEED).fit(X_train, y_train)
print(f"Random search (forest): best
{rf_search.best_params_}, CV AUC
{rf_search.best_score_:.3f}")

# ------------------------------------------------------
------- 5. Nested CV
```

```
# best_score_ above is optimistic because the same folds
picked AND scored the winner.
# Nested CV: the inner loop tunes, the outer loop gives an
honest estimate of the whole procedure.
print("\n== Step 4: nested cross-validation (unbiased
estimate of the tuning procedure) ==")
outer = StratifiedKFold(n_splits=5, shuffle=True,
random_state=SEED + 1)
inner_cv = StratifiedKFold(n_splits=3, shuffle=True,
random_state=SEED + 2)
nested = cross_val_score(
    GridSearchCV(Pipeline([("sc", StandardScaler()),
("clf", LogisticRegression(max_iter=2000))]),
                 {"clf__C": np.logspace(-3, 2, 11)},
cv=inner_cv, scoring="roc_auc"),
    X_train, y_train, cv=outer, scoring="roc_auc",
n_jobs=-1)
print(f"Nested CV AUC (logistic): {nested.mean():.3f} +/-
{nested.std():.3f}")

# ----------------------------------------------------------
------- 6. Final test (once)
print("\n== Step 5: final evaluation on the untouched test
set ==")
final = log_search.best_estimator_  # chosen using training
data only
proba = final.predict_proba(X_test)[:, 1]
print(f"Test ROC AUC: {roc_auc_score(y_test, proba):.3f}")
print(classification_report(y_test, final.predict(X_test),
target_names=["malignant", "benign"]))
```